Sample Predicted Questions

SECTION A:

Q1. (Type A – Precision Concept Check)
a) Define REST and explain the term *statelessness*.
b) Differentiate between Authentication and Authorization.

Q2. (Type A – Precision Concept Check)
a) What is MVC architecture? Explain the role of Model, View and Controller.
b) What is JSON? Why is it used in REST APIs?

Q3. Flask Pseudocode Questions

Question 1:

```
START
User sends GET request to "/hello"
IF route "/hello" exists
   RETURN "Hello Student"
ELSE
   RETURN 404
END
```

What will be the output if the route exists?

Question 2:

```
START
User sends POST request to "/login"
IF method == POST
   READ username and password
   RETURN "Login Attempted"
ELSE
   RETURN "Load Login Page"
END
```

What will be returned for:
a) GET request
b) POST request

Question 3:

```
START
Request received at "/students"
IF method == GET
   RETURN list of students
IF method == POST
   ADD new student
END
```

Which REST operations are implemented here?

Q4. (Type B – Code Reading & Debugging)
Study the following Flask code and answer the questions:

```
from flask import Flask
app = Flask(__name__)

@app.route('/hello')
def hello(name):
   return "Hello " + name

if __name__ == "__main__":
   app.run()
```

a) Identify the error in the above code.
b) Rewrite the correct route definition.
c) What will be the correct URL to pass the name dynamically?

Q5. (Type A – Precision Concept Check)
a) What is Flask Blueprints?
b) Write any two advantages of modular architecture.

Q6. (Type B – Code Reading & Output Prediction)
Observe the following code:

```
@app.route('/add', methods=['GET', 'POST'])
def add():
   if request.method == 'POST':
      return "Form Submitted"
   return "Load Form"
```

a) What will be the output if accessed using GET method?
b) What will be the output if accessed using POST method?
c) Why do we specify methods in route decorator?

Q7. (Fill in the Blanks – Flask Routing)
Complete the following Flask code:

```
from flask import Flask
app = Flask(__name__)

@app.route('/user/<_____>')
def show_user(_____):
    return "User: " + _____

if __name__ == "_____":
    app.run(debug=True)
```

Q8. (Incomplete Code – Complete & Justify)
Complete the following login verification code using Flask-Bcrypt:

```
from flask_bcrypt import Bcrypt
bcrypt = Bcrypt(app)

stored_password = bcrypt.generate_password_hash("admin123")

user_input = "admin123"

if bcrypt._____(stored_password, user_input):
    print("Login Successful")
else:
    print("Invalid Password")
```

a) Fill in the missing function.
b) Explain why hashing is important.

SECTION B:

Q9. (Type B – Code Debugging & Justification)
Analyze the following authentication code:

```
from flask_bcrypt import Bcrypt
bcrypt = Bcrypt(app)

password = "admin123"
hashed = bcrypt.generate_password_hash(password)

if hashed == "admin123":
    print("Login Successful")
```

a) Identify the logical error.
b) Explain why the comparison is incorrect.
c) Write the correct way to verify password using Flask-Bcrypt.

Q10. (Type C – Scenario-Based Reasoning)
You are developing a **Student Management System** using Flask.

Requirements:

- Only Admin can delete student records.
- Normal users can only view student details.
- System must follow REST principles.
- Authentication must be token-based (JWT).

Answer the following:
a) Which HTTP methods will you use for CRUD operations?
b) How will JWT help in securing the API?
c) Explain how role-based access control will be implemented.

Q11. (Code Analysis & Error Identification)
Analyze the following Flask API:

```
@app.route('/add/<int:a>/<int:b>', methods=['POST'])
def add(a, b):
    result = a + b
    return result
```

a) Identify two issues in the above code.
b) Rewrite the correct version.
c) What HTTP method should ideally be used for fetching calculation results and why?

Q12. (Scenario-Based Question – REST + JWT)
You are building a **Library Management REST API**.

Requirements:

- Admin can add/delete books.
- Students can only view books.
- JWT authentication must be implemented.
- API should follow REST principles.

Answer the following:
a) Which HTTP methods will be used for Add, Delete and View operations?
b) How will JWT secure protected routes?
c) How will you implement role-based access control?

Q13. (Type D – Complete Code Writing)
Write a Flask REST API to perform the following:

1. Create a Student model with id and name.
2. Implement a POST API to add a student.
3. Implement a GET API to fetch all students.
4. Secure one route using JWT authentication.

You may write necessary imports and configuration.

Q14. (Complete Code Writing – Short Application)
Write a Flask application that:

1. Creates a route /square/<int:num>
2. Returns the square of the given number in JSON format
3. Handles invalid URL access properly

Write complete runnable code.